Phase 1: developer handoff

Delivered 4 October 2026. Read alongside DEVELOPER-GUIDE.md, RUNTIME-DEVELOPER-GUIDE.md and the enterprise roadmap. This document describes the implemented security slice, not the full target architecture.

Module boundaries

Module	Responsibility
`backend/app/access/models.py`	Workspaces, bootstrap state, principals, credentials, rate windows, policies, repository registrations, grants and audit events
`access/core.py`	Scrypt passwords, token digests, CSRF derivation, scoped sessions, bootstrap state and durable audit chaining
`access/middleware.py`	Authentication, permissions, CSRF, request-intent audit and bounded security request bodies
`access/schemas.py`	Strict request contracts, permission/path validation and policy limits
`access/gateway.py`	Read-only pinned Git access; explicit scopes, protected paths, file/byte caps and audit-before-release
`access/routes.py`	Auth/session administration, workspaces, policy, repositories, source requests and audit queries
`backend/app/security.py`	Existing browser origin/host boundary; separate from account authorization
`frontend/src/security/AccessShell.tsx`	First-run setup, login, logout, expiry handling and identity shell
`frontend/src/security/SecurityConsole.tsx`	Permission-aware administration, password change, source requests and audit
`backend/alembic/versions/0003_security.py`	Additive security schema and audit immutability triggers

Static algorithms and runtime mapping/fusion stay in their existing service packages. HTTP handlers use ScopedSession; internal worker/retention processes deliberately use ordinary sessions. Do not add raw SQL or an unscoped session to a request handler without an explicit tenant authorization design. SQLAlchemy scoping is application enforcement, not PostgreSQL row-level security.

Request and identity flow

Public endpoints are health, readiness, setup status, bootstrap and login. POST bootstrap/login require `X-ArchIntel-Request: 1`. Management endpoints and OpenAPI/docs require an identity. Browser sessions use an opaque HttpOnly, SameSite=Strict cookie. Unsafe cookie requests require `X-CSRF-Token`, derived from the session token; `/security/me` returns it to the authenticated browser. It lives in frontend memory. CORS permits configured origins and the required headers.

Passwords use scrypt with a random 16-byte salt, N=131072, r=8 and p=1. Unknown usernames still perform hash work. Login/password attempts have persisted rolling 15-minute limits (8 per username, 40 per client host). Browser credentials default to 8 hours, configurable from 1 to 24. Service credentials last 168 hours. Only hashes of opaque credentials are stored. Service tokens cannot act as human-cookie sessions and human session tokens cannot act as service bearer tokens.

Password changes verify the current password, remove all credentials for the principal, hash the new password and issue a replacement session atomically with audit. Revoking a principal invalidates credentials and active state. No SSO, forgotten-password recovery,

automatic credential renewal or role catalog is implemented yet.

Runtime ingestion is an explicit exception: a bearer token is checked against the exact runtime session, which supplies the tenant. It authorizes ingestion only. Runtime session state checks remain in the existing runtime routes/services. Management and telemetry credentials are not interchangeable.

Tenant enforcement

Authenticated tenant identity determines ScopedSession.info. SQLAlchemy loader criteria constrain Identity models, tenant audit rows and RuntimeEvent rows through their parent session. Flush validation assigns the authenticated tenant to new Identity rows and rejects cross-tenant mutations. Parent resource lookups prevent indirect ID escapes. Analysis admission deliberately uses an internal unscoped count to preserve the shared four-job capacity limit; it exposes no other workspace records.

The installation owner may provision a workspace plus a distinct administrator, but remains scoped to the local workspace for data access. Usernames are installation-wide unique. Existing tenant_id=local records remain local; migration does not rewrite analyses or runtime snapshots.

Source authorization pipeline

1. Middleware authenticates the caller and requires SOURCE_READ; intent is committed.
2. Under the security RLock, the gateway rechecks the current principal's active state/permission and loads the tenant-scoped repository, latest policy and explicit grant.
3. Require SOURCE_READ, LOCAL_CONTEXT, enabled policy and the exact registered 40-character commit. Protected paths take precedence over literal allowed files/prefixes.
4. Resolve the checkout strictly below an operator-configured source root; reject symlinks/junctions and linked Git directories.
5. Use argument-list Git subprocesses with no shell, no pager, no replacement objects, no inherited GIT_* overrides, no global/system Git config, no prompts and a 15-second timeout. Read only regular committed UTF-8 blobs, never working-tree files or submodule/link targets. Enforce file and aggregate-byte limits.
6. Close the read transaction, commit an ALLOW/DENY metadata event, and only then return the complete context. Audit failure returns no source. There is no partial-file success.

The RLock serializes source release with policy/grant/principal mutations **within one API process**. PostgreSQL row locks also serialize audit heads and policy revisions. Multiple API workers are not supported for this security release; cross-process authorization fencing must precede scale-out. Trusted server operators can modify checkout/object storage and the database; this is not an OS sandbox against a malicious operator.

Policy and audit

Policies are append-only revisions through the API with optimistic expected_revision checking. Current outcomes are ALLOW/DENY. Limits allow up to 20 files and 262,144 bytes but default to 5/65,536. External transfer is fixed DENY and telemetry fixed DISABLED. Phase 2 adds data classification, sensitive-content filtering, redaction and destination/model policy; current filename blocking is not a secrets detector.

Workspace audit events contain sequence, previous hash, canonical-JSON event hash, actor, action, outcome, resource, timestamp and request ID. Source events include requested/permitted/denied paths, purpose, commit, policy revision, destination and byte count, never source content. Ordinary security changes and their audit commit in one transaction. General analysis/runtime writes record REQUESTED intent before dispatch; this is not a comprehensive atomic business-action outcome log. Malformed requests rejected before the gateway do not have a source manifest decision.

SQLite and PostgreSQL triggers prohibit ordinary audit UPDATE/DELETE. Verification checks all events against the workspace head. A database owner can remove triggers and replace the entire chain; no external immutable anchor exists yet. Logging reports exception

type without SQL parameters or credential-bearing exception text. Source failure responses are generic. Keep free-text audit purposes free of secrets.

API summary

All paths below begin `/api/v1/security`.

Endpoint	Behavior
GET status; POST bootstrap/login	Public setup/login; POST client header required
GET me; POST logout/password	Authenticated identity/session/password operations
GET/POST principals; POST principals/{id}/revoke	Workspace identity management
POST workspaces	Installation-owner-only provisioning
GET/POST policy	Read/latest revision and optimistic update
GET/POST repositories	Register pinned, server-local checkouts
POST grants	Replace exact principal/repository scope
POST context	Bounded read through gateway
GET audit?after=N; GET audit/verify	Paginated events and hash verification

Use a service bearer token for automation and supply only its explicit permissions. Cookie-based clients must obtain the CSRF token from `/me` and supply it on unsafe requests. There is no authentication-disable test switch.

Migration and deployment

Startup upgrades Alembic to 0003, then initializes bootstrap state before recovering the analysis worker. The schema adds nine security tables and a local workspace. Existing static/runtime records are untouched by migration. Migration 0003 currently creates tables from the security model definitions; freeze or explicitly version those definitions before future incompatible model edits.

`SECURE_COOKIES=false` supports the prepared loopback HTTP launcher. HTTPS deployments require `SECURE_COOKIES=true`, explicit host/origin configuration and deployment review. Source roots default to DATA_DIR/source-repositories; configure SOURCE_REPOSITORY_ROOTS as a JSON array. Bootstrap key files use restrictive creation mode where supported; Windows operators must ensure the inherited filesystem ACL is appropriate. Never include .data or .env in version control or support bundles.

Use one API process. Multi-host workers, distributed locks, production IdP/TLS provisioning, secret-vault integration, backup/restore certification and OS execution isolation remain future work. The pre-migration local row snapshot is preservation evidence, not a tested disaster-recovery backup.

Validation commands

From the project root, PowerShell:

```
$phaseTests = '.data/phase1-check-' + [guid]::NewGuid().ToString('N')
$env:ARCHINTEL_RUN_SCANNER_TESTS = '1'
.\.runtime\python\tools\python.exe -m pytest backend/tests -q -p no:cacheprovider --basetemp=$phaseTests
Set-Location frontend
```

```
npm.cmd test
npm.cmd run build
npx.cmd playwright test --config playwright.security.config.ts
```

The security browser configuration creates a separate SQLite database under a unique .data/browser-validation-* directory, uses port 18001 and the prepared Java scanner, and never consumes the real installation bootstrap. Browser test account passwords are fixture values only. Do not point these tests at a customer server. The existing runtime browser suite relies on an older persisted PetClinic fixture and is not included in the isolated security suite; runtime API/isolation regressions remain in pytest.

Next extension boundary

Phase 2 should add the AI gateway, local adapter, minimized/filtered context, policy routing, output validation and audited dispatch. Direct OllamaProvider.select_facts now raises instead of issuing network calls. Deterministic static/runtime evidence stays available. New agents must not read source or contact models around the gateways. Do not start Phase 2 until the Phase 1 user verification checkpoint is accepted.

Operator live-check scripts now require ARCHINTEL_SERVICE_TOKEN from a scoped service identity. verify_delivery.py needs ANALYSIS_READ. validate_runtime_petclinic.py is a trusted local maintenance utility with database access and requires a local-workspace service identity with ANALYSIS_READ, ANALYSIS_WRITE, RUNTIME_READ and RUNTIME_WRITE; it verifies these before seeding its existing local fixture or launching Java. No new real PetClinic run is claimed in this phase.